

Trabalho Final - Arquitetura de Computadores

Análise e Simulação de um Pipeline MIPS32

Marcus Sebastião Adriano Rocha Neto¹, Henrique Vilella², and Alexandre Sbrissia³

¹Informática Biomédica, Universidade Federal do Paraná (UFPR)

²Informática Biomédica, Universidade Federal do Paraná (UFPR)

³Ciência da Computação, Universidade Federal do Paraná (UFPR)

Dezembro de 2025

Orientador:

Prof. Eduardo Todt

Universidade Federal do Paraná (UFPR)

Resumo

Este documento detalha o projeto e a implementação de um simulador para o pipeline MIPS de cinco estágios, desenvolvido como ferramenta didática para o estudo de Arquitetura de Computadores. O software, construído em Python com interface gráfica em Tkinter, analisa um conjunto de instruções MIPS32 fornecido em formato assembly (.asm). O foco da simulação reside na detecção de conflitos de dados (*data hazards*) e na aplicação de mecanismos de resolução, como a inserção de paradas no pipeline (*stalls*) e o uso de adiantamento (*forwarding*). O resultado é a geração de um diagrama de execução ciclo a ciclo, que visualiza o fluxo das instruções e a ocorrência desses eventos, sem, no entanto, realizar a execução aritmética das operações. A arquitetura modular do projeto, que separa a análise sintática, a simulação do pipeline e a renderização da interface, é também discutida.

Palavras-chave: Arquitetura de Computadores, MIPS32, Pipeline, Simulação, Conflitos de Dados.

1 Introdução

Este trabalho apresenta o desenvolvimento de um simulador de pipeline MIPS32, capaz de gerar o diagrama de execução de um conjunto de instruções, demonstrando conceitos fundamentais de Arquitetura de Computadores, como *data hazards*, *stalls* e *forwarding*. O objetivo é modelar o fluxo de instruções através do pipeline de 5 estágios, sem executar as instruções de fato, mas analisando suas dependências e interações.

2 Visão Geral

O projeto consiste em um simulador do pipeline MIPS de 5 estágios, implementado em Python. Utilizando a biblioteca *Tkinter*, o programa cria uma interface gráfica que exibe o diagrama de execução gerado com base nas instruções fornecidas pelo usuário. A simulação não manipula valores da memória (instruções como *sw* e *lw*), mas modela o avanço das instruções pelos estágios do pipeline, realizando detecção de *hazards* de dados, inserção de *stalls* (bolhas) e aplicação de *forwarding* quando possível.

3 Arquitetura do Projeto

O projeto foi estruturado em módulos com responsabilidades bem definidas, conforme descrito a seguir:

1. `parser.py` (Tradutor)

Responsabilidade: Ler um arquivo `.asm` e converter cada linha em um objeto `MipsInstruction`.

Funcionamento: Realiza a análise sintática (*parsing*) utilizando expressões regulares para extrair o mnemônico e seus operandos. Classifica cada instrução conforme o mapa `INSTRUCTION_MAP` e armazena seus campos (`rs`, `rt`, `rd`, `imm`) em uma estrutura de dados, tratando comentários e linhas vazias.

2. `hazard_detect.py` (Simulação)

Responsabilidade: Simular ciclo a ciclo o fluxo de instruções no pipeline, detectando *data hazards*.

Funcionamento: Controla os estágios ativos do pipeline, identifica dependências entre instruções e determina se deve aplicar *stall* ou *forwarding*, gerando a matriz final que representa o diagrama de execução.

3. `diagram.py` (Diagrama)

Responsabilidade: Renderizar o diagrama de execução usando *Tkinter*.

Funcionamento: Gera uma janela para desenhar uma grade representando ciclos e estágios. Cada célula recebe formatação distinta conforme o estágio (IF, ID, EX, MEM, WB, Stall etc.).

4. `main.py`

Responsabilidade: Coordenar a interação entre os módulos, inicializar a aplicação e controlar o fluxo geral de execução.

4 Dificuldades Enfrentadas e Lógica do Projeto

Primeiramente, a escolha da linguagem da implementação, Python, se dá pela biblioteca gráfica *Tkinter*, que oferece mais recursos gráficos, possibilitando a construção do diagrama do pipeline. A maior dificuldade enfrentada foi a construção da simulação, pois, para obtermos um

diagrama com informações coerentes era preciso saber, por exemplo, o valor de cada registrador, para assim saber se o desvio condicional é atendido ou não. Com isso, implementamos, no arquivo *hazard_detect.py*, tanto uma lista que percorre as cinco instruções dentro do pipeline quanto uma lista que guarda o histórico de instruções para a visualização do código.

5 Conclusão

Este trabalho apresentou a implementação de um simulador visual para o pipeline MIPS de cinco estágios. A ferramenta foi capaz de analisar códigos em Assembly e gerar diagramas de execução ciclo a ciclo, ilustrando corretamente o tratamento de Data Hazards via stall e forwarding, bem como Control Hazards via flush.

A decisão de implementação da estratégia Always Not Taken, além do cálculo das condições de desvio, garantiu a corretude da simulação em estruturas de controle como loops. Embora a memória de dados tenha sido modelada de forma simplificada, o simulador atende ao propósito de visualizar o comportamento do pipeline, servindo como uma ótima ferramenta para o estudo de Arquitetura de Computadores.

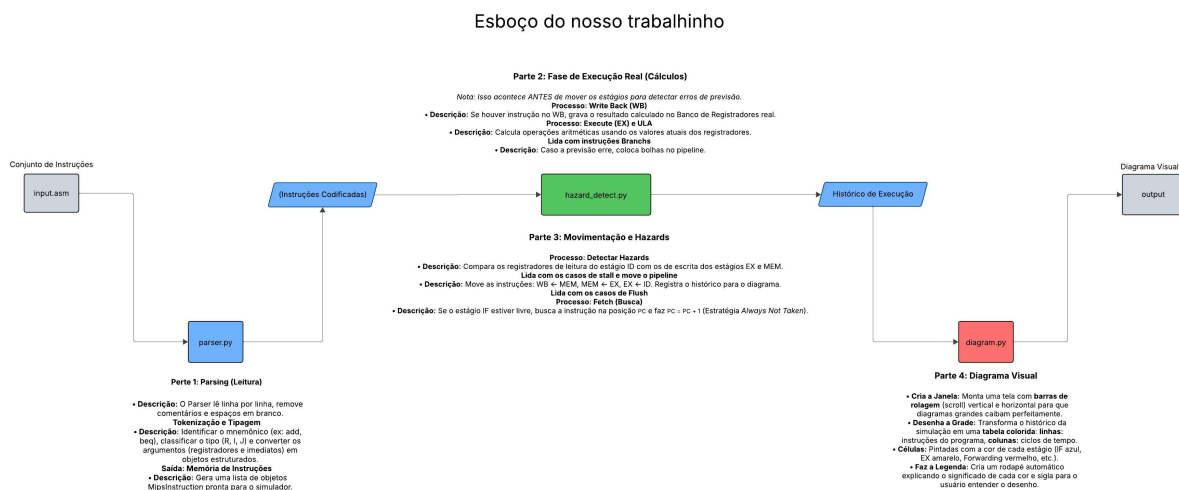


Figura 1: Esboço de Execução

	Ciclo 1	Ciclo 2	Ciclo 3	Ciclo 4	Ciclo 5	Ciclo 6	Ciclo 7	Ciclo 8	Ciclo 9	Ciclo 10	Ciclo 11	Ciclo 12	Ciclo 13	Ciclo 14	Ciclo 15	Ciclo 16
addi \$0, \$zero, 5 # Inicializa \$0 com 5	IF	ID	EX	MEM	WB											
addi \$1, \$zero, 10 # Inicializa \$1 com 10		IF	ID	EX	MEM	WB										
bit \$0, \$1, 2 # Se \$0 < \$1, pula 2 instruções a frente			IF	ID	EX	MEM	WB									
addi \$2, \$zero, 999 # \$2 = 999 (não deve executar)				IF	FLUSH											
j 3 # Jump para fim (não deve executar)					FLUSH											
addi \$3, \$zero, 100 # \$3 = 100 (marca que branch foi taken)						IF	ID	EX	MEM	WB						
addi \$0, \$zero, 15 # \$0 = 15							IF	ID	EX	MEM	WB					
addi \$1, \$zero, 8 # \$1 = 8								IF	ID	EX	MEM	WB				
bit \$0, \$1, 2 # Se \$0 < \$1, pula 2 instruções (não deve putar)									IF	ID	EX	MEM	WB			
addi \$4, \$zero, 200 # \$4 = 200 (marca que branch não foi taken)										IF	ID	EX	MEM	WB		
addi \$5, \$zero, 300 # \$5 = 300											IF	ID	EX	MEM	WB	
addi \$v0, \$zero, 1 # Instrução final												IF	ID	EX	MEM	WB

Figura 2: Exemplo de Saída